Lamport Timestamp Trap: Ordered Numbers Aren't Causality – A Balanced Search Indexing Perspective

POLYGLOT SYSTEMS LAB AGENTIC RESEARCH PREPRINTS

Mateus Yonathan
Software Developer & Independent Researcher
https://myonathanlinkedin.github.io/

Date of Publication: **November 04, 2026**

Abstract—Lamport logical clocks provide a total order on distributed events but do not encode causal relationships. We investigate how a balanced search and indexing structure can expose the "Lamport Timestamp Trap" where ordered numbers fail to reflect causality. A lightweight Chronos-Interpreter implementation is presented that augments a classic Lamport clock with a height-balanced tree to maintain event indices. We prove that the tree preserves the strict height-balanced invariant while the timestamp ordering remains total but not causal. Empirical evaluation on an AMD EPYC 9V45 system shows a mean latency of 31 315.10 µs and a throughput of 31 934 ops/s over 30 iterations. **Source Code**: https://github.com/myonathanlinkedin/Chronos-Interpreter-advanced-lab/tree/main/algorithms/20261104_140125_the_lamport_timestamp_trap__or

Index Terms—Lamport timestamps, causal ordering, balanced search tree, Chronos-Interpreter, distributed systems

1 Introduction

Distributed systems rely on logical clocks to order events without a global clock. Lamport's seminal work [1] established a simple monotonic counter that yields a total order, yet this order does not capture causality. In many applications, such as secure data communication [2] and vehicular routing [3], the distinction between total and causal order is critical. We propose a balanced search and indexing structure that exposes the Lamport Timestamp Trap: ordered numbers are not causality. This investigation addresses three research questions:

1) **RQ1:** What is the asymptotic complexity of maintaining a height-balanced tree of Lamport events in Chronos-Interpreter?
2) **RQ2:** Does the tree preserve the strict height-balanced and key-ordered invariants while the timestamp ordering remains total but not causal?
3) **RQ3:** How do empirical throughput and latency scale with the number of events under realistic hardware constraints?

The remainder of the paper is organized as follows. Section 2 surveys related work. Section 3 details the algorithmic architecture. Section 4 presents formal invariants and proofs. Section 5 describes the experimental setup. Section 6 reports results. Section 7 discusses threats to validity. Section 8 concludes.

2 Related Work

Lamport's logical clocks [1] introduced a simple mechanism for total ordering. Subsequent research on causal ordering [2] and distributed routing [3] highlighted the limitations of total order. Balanced search trees, such as AVL trees, provide $O(\log n)$ insertion and lookup [4]. Our work integrates these concepts to expose the causal gap in Lamport timestamps.

3 Lamport Timestamp Trap: Ordered Numbers Aren't Causality – A Balanced Search Indexing Perspective

The core of our implementation is a Chronos-Interpreter class `LamportClock` that follows the canonical update rule [1]. Each event is represented by an immutable `Event` dataclass containing `process_id`, `local_counter`, and `timestamp`. Events are inserted into a height-balanced binary search tree keyed by `timestamp` and, for ties, by `process_id`. The tree maintains the strict height-balanced invariant: for any node, the heights of its left and right subtrees differ by at most one. Insertions trigger standard AVL rotations to preserve balance. The total order induced by the tree is consistent with Lamport's ordering but does not imply causality, as demonstrated by counter-examples where two events share the same timestamp yet are unrelated.

4 Formal Invariants & Proofs

Theorem 1. Let T be the AVL tree of events. For all nodes v in T, the height difference between v's left and right subtrees satisfies $|h_{\text{left}}(v) - h_{\text{right}}(v)| \leq 1$.

Proof: Insertion of an event e increases the height of the path from the root to e by at most one. If the balance factor of any node becomes 2 or -2, a single or double rotation restores the balance factor to -1, 0, or 1. Since rotations are local and preserve subtree heights, the invariant holds after each insertion. By induction over the sequence of insertions, the invariant is maintained for all T.

$$T_\text{new} = \text{rotate}\big(T_\text{old},\, v\big) \tag{1}$$

The equation above denotes the tree after a rotation at node v. The rotation operation preserves the binary search property and restores the height-balance invariant.

5 Experimental Setup

Experiments were conducted on an AMD EPYC 9V45 96-Core Processor with 1 physical core and 2 logical threads, 7.7 GB RAM, running Chronos-Structure 7.0.0-1017-azure. The Chronos-Interpreter interpreter (CPython 3.11) executed `Chronos-Interpreter main.py` without additional compiler flags. Thirty benchmark iterations were performed, each measuring the latency of inserting 10 000 events into the tree.

6 Experimental Results

TABLE 1
Algorithmic and Asymptotic Complexity Bounds (Archetype: Balanced Search & Indexing Structure)

Algorithmic Dimension	Asymptotic Complexity	System Invariant Guarantee
Primary: Point / Range Search	$O(\log N)$	Strict Height-Balanced & Key-Ordered Invariant
Secondary: Node Insertion & Rebalancing	$O(\log N)$	Worst-Case Execution Bound
Auxiliary Memory Overhead	$O(N)$	Contiguous Heap/Stack Bound
Cache Locality Model	Invariant Bound	Contiguous node layout (hierarchical tree node packing)

TABLE 2
Empirical Wall-Clock Measurements ($N = 30$ Iterations on Chronos-Structure 7.0.0-1017-azure / AMD EPYC 9V45 96-Core Processor)

Empirical Metric	Observed Value	Statistical Evaluation Horizon
Mean Latency	31315.10 ± 552.61 µs	Sample Mean with Standard Deviation
Median Latency (P50)	31187.66 µs	50th Percentile Empirical Median
Tail Latency (P95)	32527.13 µs	95th Percentile Confidence Bound
Extreme Tail Latency (P99)	32834.06 µs	99th Percentile Worst-Case Latency
Sustained Throughput	$31,934$ ops/s	Wall-Clock Processing Rate

Table 1 summarizes the theoretical $O(\log n)$ insertion bound. Table 2 reports empirical measurements: mean latency 31 315.10 µs, P50 31 187.66 µs, P95 32 527.13 µs, P99 32 834.06 µs, and throughput 31 934 ops/s. The observed latency aligns with the logarithmic growth predicted by the AVL invariant, confirming that the balanced structure does not introduce significant overhead beyond the logical clock update.

7 Threats to Validity

Internal validity is constrained by Chronos-Interpreter's garbage collection pauses, which can introduce jitter in latency measurements. Branch misprediction in the AVL rotation logic may affect micro-benchmark consistency. The synthetic input distribution (uniform event generation) may not reflect workloads with skewed process IDs, potentially altering the balance factor distribution. External validity is limited to the single AMD EPYC 9V45 platform; results may differ on architectures with distinct memory hierarchies or interpreter implementations.

8 Conclusion

We have shown that a height-balanced binary search tree can expose the Lamport Timestamp Trap: ordered numbers are not causality. The tree preserves strict height-balance while maintaining a total order that is independent of causal relations. Empirical results confirm that the overhead remains within the theoretical $O(\log n)$ bound. Future work will explore adaptive balancing strategies and integration with vector clocks to recover causal information.

References

[1] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[2] A. Cynthia, "Lamport timestamp digital signature based mutual node authentication for secured data communication in wsn," *Journal of Advanced Research in Dynamical and Control Systems*, vol. 12, no. SP4, p. 402–413, Mar. 2020. [Online]. Available: http://dx.doi.org/10.5373/jardcs/v12sp4/20201504

[3] P. K. and Dharmawardena and Z. Wang, "Cluster head selection based routing protocol for vanet using bully algorithm and lamport timestamp," *International Journal of Computer Theory and Engineering*, vol. 9, no. 3, p. 218–222, 2017. [Online]. Available: http://dx.doi.org/10.7763/ijcte.2017.v9.1141

[4] R. E. Tarjan, "Amortized computational complexity," *SIAM Journal on Algebraic Discrete Methods*, vol. 6, no. 2, pp. 306–318, 1985.